

One day a boy came with a story.

assigning a number {word based tokenization}

{~~word~~ character based tokenization}

Vocabulary → Token
→ Token Id

they make a huge input file. Hard for the transformer to read.

dataset → Byte Pair Encoding Algorithm (Tokenizer) → Intermediate {Subword Tokenization}

Token

Token Id

Vocabulary

→ characters (every characters)
→ words (common, repeated)
→ subwords (rarely used)

Our goal is to do 2 thing

Tokenize data

↓
GPT-2 subword tokenizer (BPE)

tokens like

[442, 518, 213, 456]

[id: [442, 518, 213, 456],
'len': 5]

↓
Apply on full data

↓
store all the token IDs in .bin file (cause stores directly on disk)

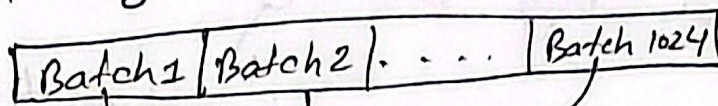
↓
create a .bin file

↓
Create a memory mapped array

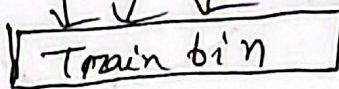
↓
split dataset into batches and add everything to train .bin

Febustat®

2.12M story $\rightarrow$ 1024 batch



Token IDs are collected



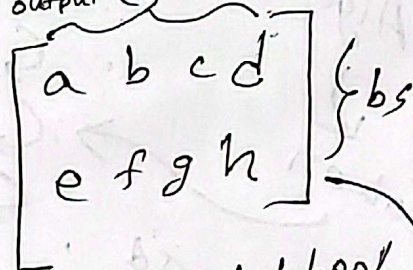
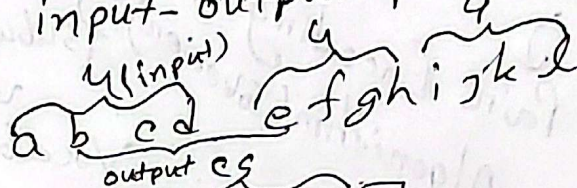
memory mapped array
or `np.memmap()`
stores data in disk

Creating input-output pair

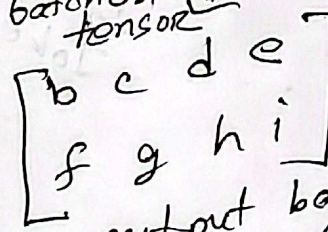
context size = 4

batch size = 2

output is input shifted
one in right



Input batches tensor

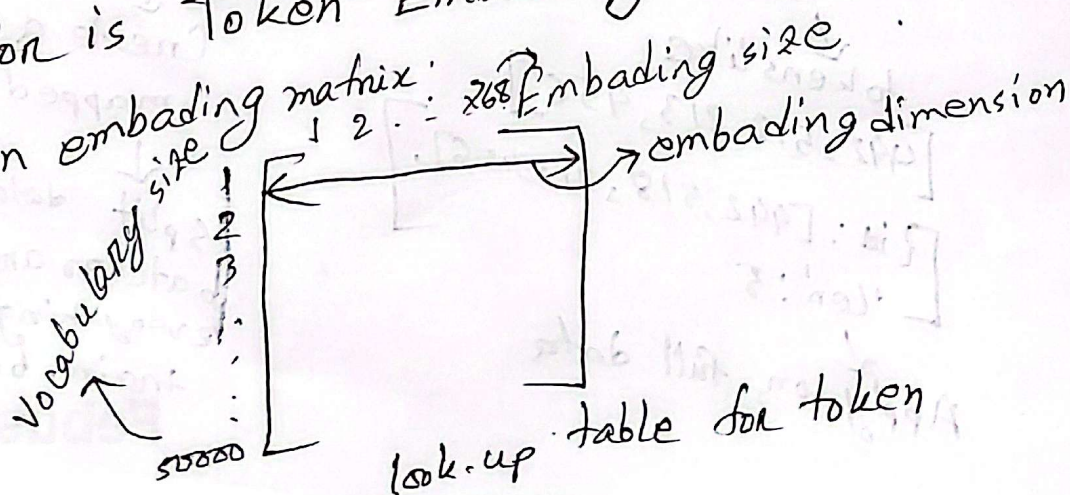
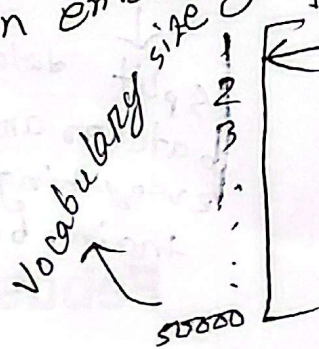


output batches tensor

Token = $\begin{bmatrix} \text{id}: [442, 10, 11, 12] \\ \text{len}: 4 \end{bmatrix}$

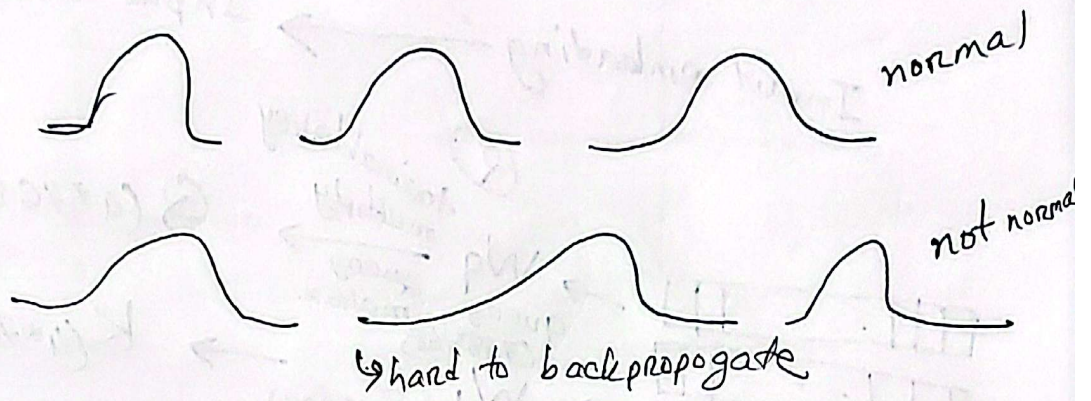
* converting a token ID into a higher dimension vector is Token Embedding

Token embedding matrix: $1 \ 2 \ \dots \ 268$ Embedding size



look-up table for token

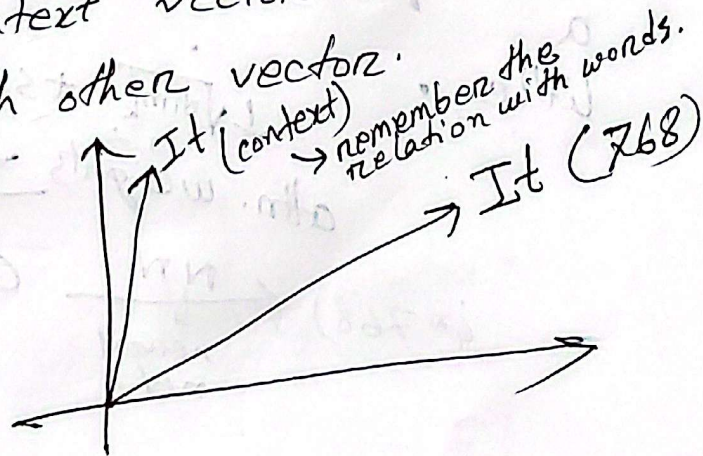
Layer normalization decrease internal covariant shift.



Multi head attention

The dog chased a ball. It could not chase it.

attention augments a value to know about neighbour
attention ~~can~~ maintain a value as it is. But it
creates a context vector of that value. Context
correlate with other vector.

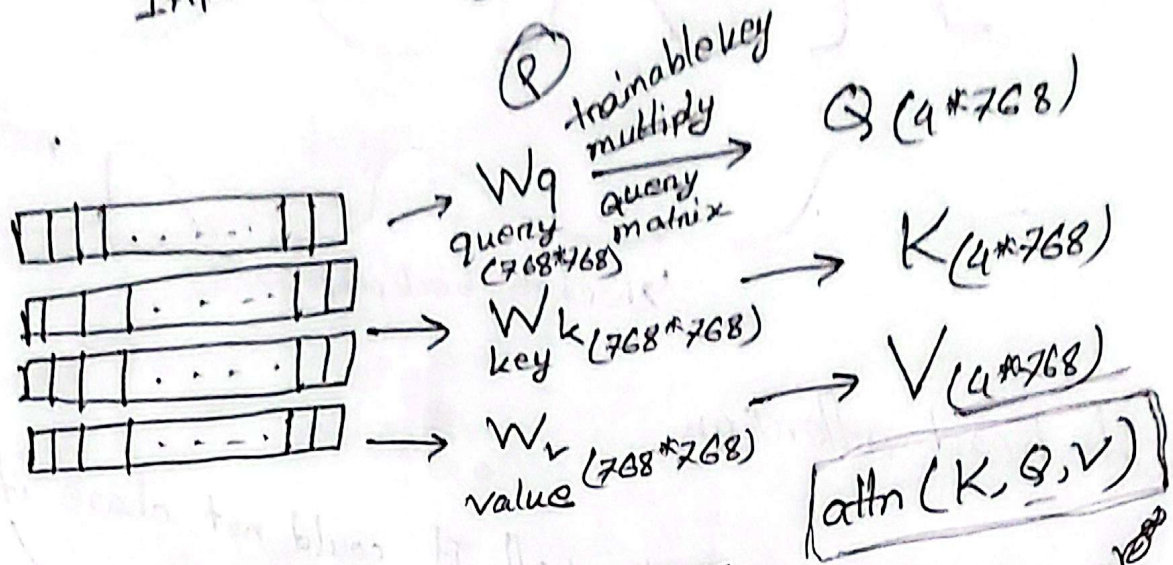


multihead attn. collects the attn. scores with neighbour

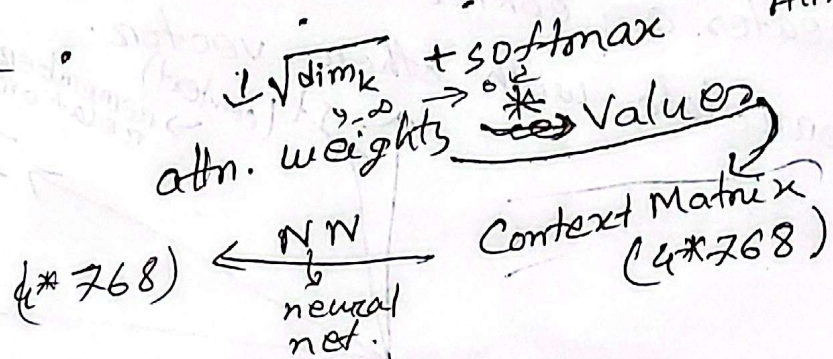
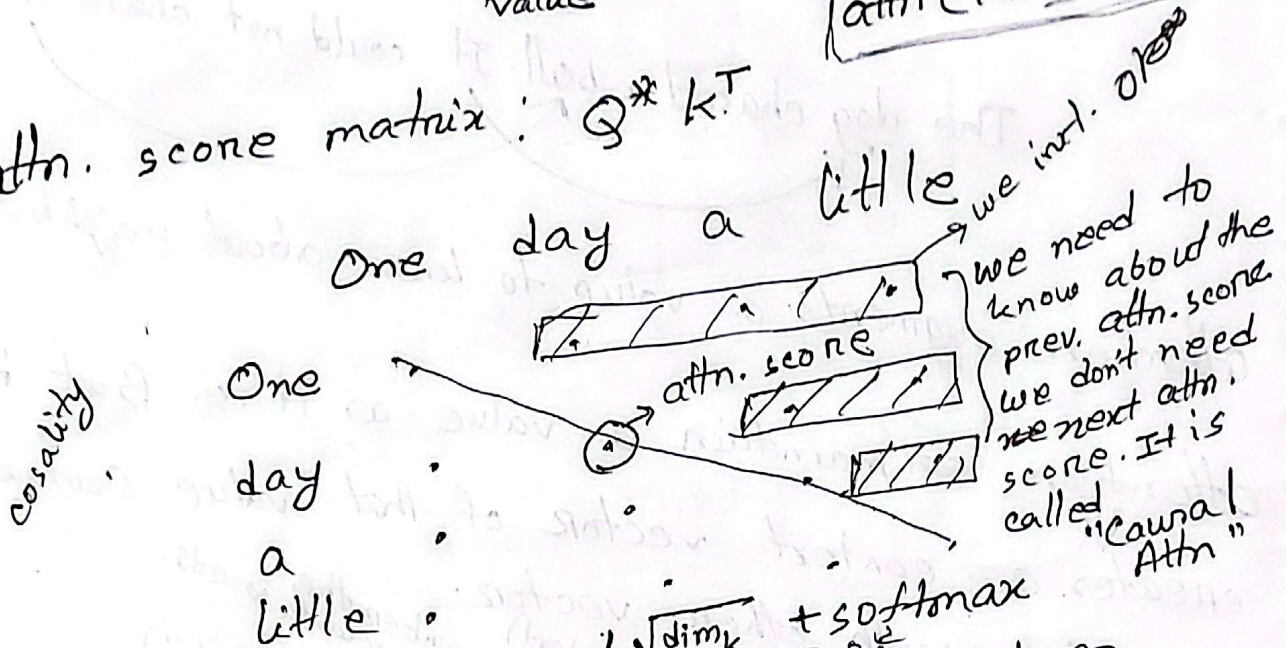
Febustat®

After multihead attn.

Input embedding $\rightarrow$ Input context

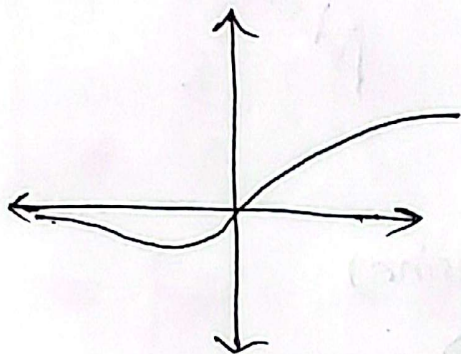


attn. score matrix: $Q \times K^T$

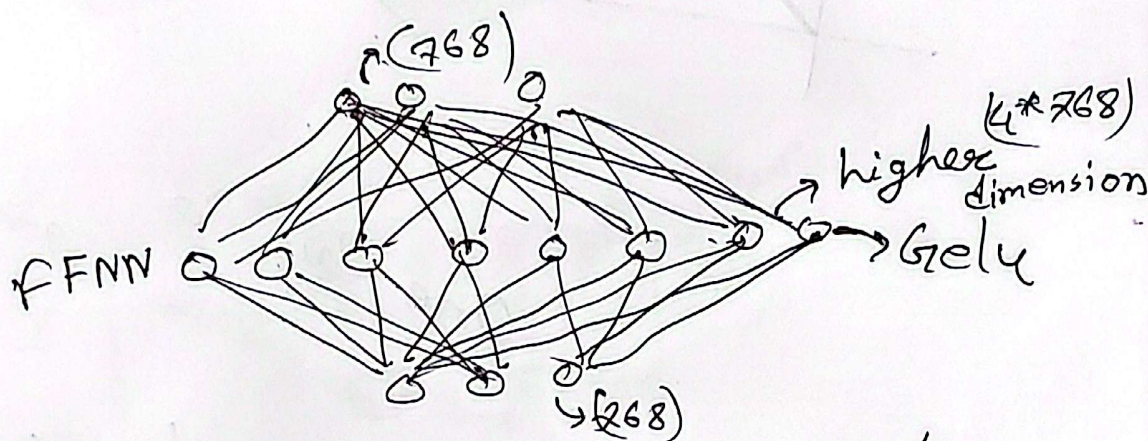
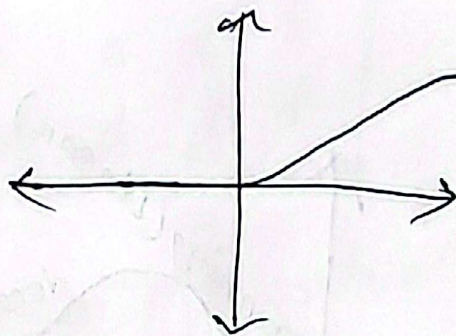


In L2 we keep $\mu = \frac{\mu}{\sigma}$ mean = 0, variance = 1 $\rightarrow$ Feed forward Neural Network

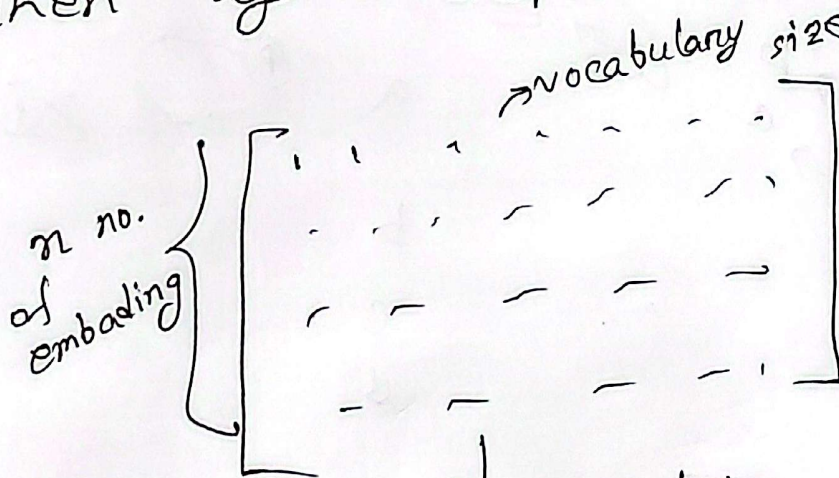
Gelu :



Relu :



Then again dropout randomly.



$$\begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \end{bmatrix}$$

Loss

$$-\frac{1}{4} [\log p_1 + \log p_2 + \log p_3 + \log p_4]$$

negative log likelihood loss

Febustat®